Big Data (MEI)

Distributed File Systems

João Pedro Costa
jcosta@isec.pt

1

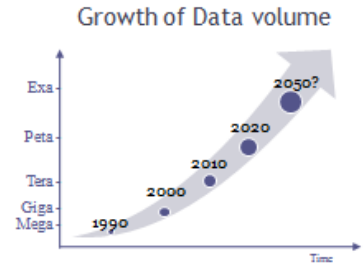
Agenda

- O aumento de dados
  - Maior número de ficheiros
  - Ficheiros maiores
- Problemas ..
  - Como e onde guardar?
  - Como processar?
- google research
  - GFS - Google File system
  - HDFS – Hadoop Distributed File System

2

Data Volume

- Increasing
- Imagine the volume



3

Stunning Data Growth Statistics

source: https://techjury.net/blog/how-much-data-is-created-every-day/

- **306 billion emails** are sent every day
- **500 million tweets** are made every day
- **95 million photos and videos** are shared every day on Instagram
- **+4,54 billion internet users** worldwide
- there are only about **200 million active** websites
- more than **1,5 billion websites** on the internet are inactive
- there are **56,5 billion web pages** indexed through Google
- in 2020, every person create **1,7 MB** of data **every second**
- **90%** of the world's data has been created in **the last 2 years**
- In 2021, every day, humans produced **2,5 quintillion** bytes of data
  (2 500 000 000 000 000 000)
- In 2025, every day, humans will generate **3 463 exabytes** of data
  (3 463 x 1024 x 1024 x 1024 gbytes)
- By the end of 2020, entire digital data was around **44 zettabytes**
  (44 x 1024 x 1024 x 1024 x 1024 gbytes)

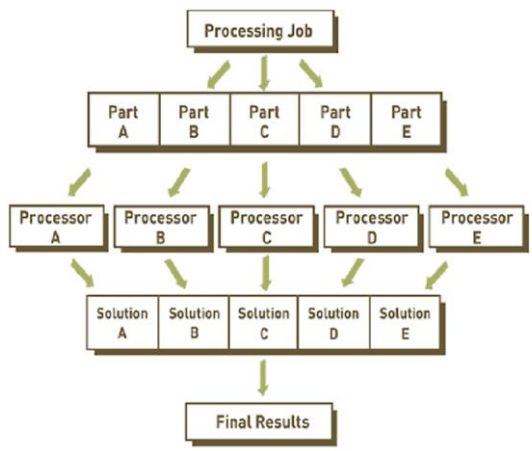
4

How to handle this?

- Infrastructure
  - Scale up
  - Scale out
  - Distributed processing infrastructures
- Storage
  - How and where to store?
  - How can processing nodes access it?
- Computation
  - Parallel processing
  - Distributed processing

5

Parallel Processing



6

Google Context

Scrap websites

Store pages

Process, Index and pageRank



How many files?

Amount of data?

How many clients? Searches/minute?

How many servers?

9

Google Context



How do you do it?

Propose solution!

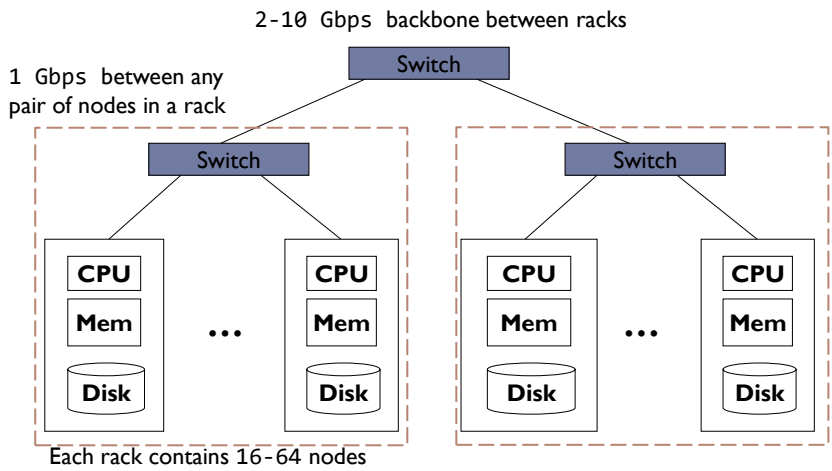
10

Motivation: Google Example

- 20+ billion web pages x 20KB = 400+ TB
- 1 computer reads 30-35 MB/sec from disk
  - ~4 months to read the web
- ~1000 hard drives to store the web
- Takes even more to **do** something useful with the data!

- **a standard architecture for such problems emerged**
  - Cluster of commodity Linux nodes
  - Commodity network (ethernet) to connect them

Cluster Architecture



In 2011 it was guestimated that Google had 1M machines, http://bit.ly/Shh0RO



13

Large-scale Computing

- for **data mining** problems on **commodity hardware**
- **challenges:**
  - **How do you distribute computation?**
  - **How can we make it easy to write distributed programs?**
  - **Machines fail:**
    - One server may stay up 3 years (1000 days)
    - If you have 1000 servers, expect to lose 1 **every day**
    - estimated Google had **~1M** machines in 2011

      ➔1000 machines fail every day!

14

Fault-tolerance

- And what about HD Disks?
  - Several disks fail per hours
- Availability ?
- Recoverability ?
- Non-Volatile storage ?
- Replication ?
- Consistency: Strong ? Weak?

15

Idea and Solution

- **Issue: Copying data over a network takes time**
- **Idea:**
  - Bring computation close to the data
  - Store files multiple times for reliability
- **Map-reduce** addresses these problems
  - Google's computational/data manipulation model
  - Elegant way to work with big data
  - **Storage Infrastructure – File system**
    - Google: GFS. Hadoop: HDFS
  - **Programming model**
    - Map-Reduce

16

Storage Infrastructure

- **Problem:**
  - If nodes fail, how to store data persistently?
- **Answer:**
  - **Distributed File System:**
    - Provides global file namespace
    - Google GFS; Hadoop HDFS;
- **Typical usage pattern**
  - Huge files (100s of GB to TB)
  - Data is rarely updated in place
  - Reads and appends are common

17

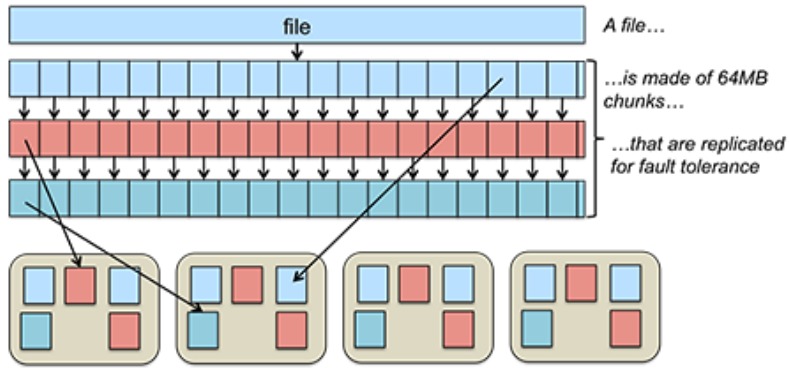
Distributed File System

- **Chunk servers**
  - File is split into contiguous chunks
  - Typically, each chunk is 16-64MB
  - Each chunk replicated (usually 2x or 3x)
  - Try to keep replicas in different racks
- **Master node**
  - Stores metadata about where files are stored
  - Might be replicated
  - a.k.a. NameNode in Hadoop's HDFS

18

GFS

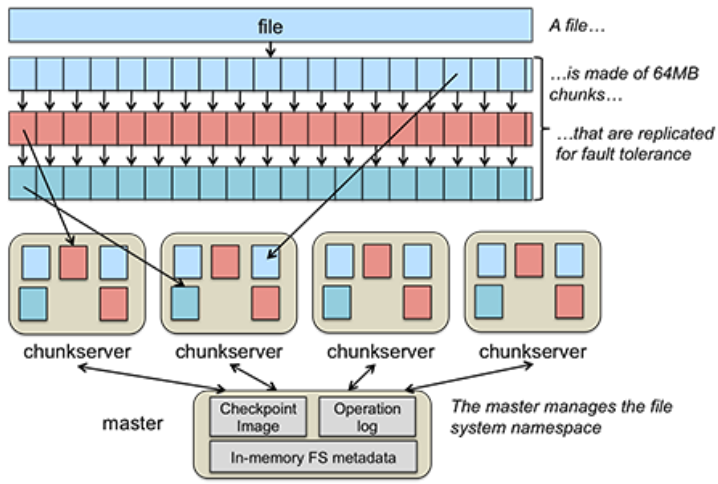
- **Each file is divided in chunks, typically of 64MB**



Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung, The Google File System. Google, SOSP'03, October 19–22, 2003.

19

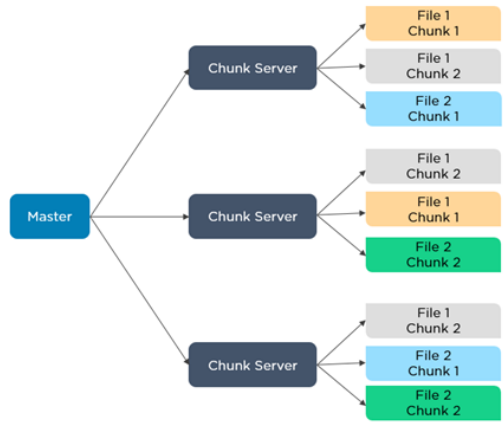
GFS



Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung, The Google File System. Google, SOSP'03, October 19–22, 2003.

20

GFS



Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung, The Google File System. Google, SOSP'03, October 19–22, 2003.

21

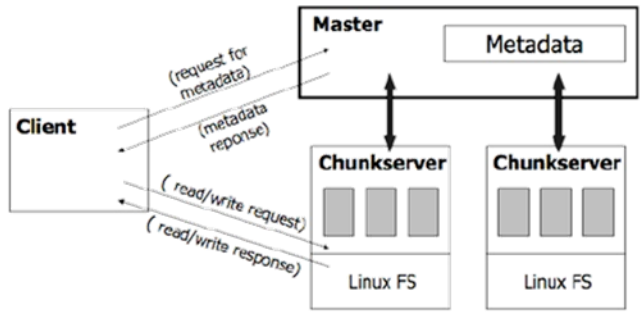
GFS Replication



22

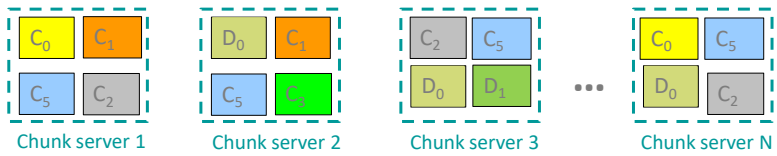
Client Interaction

- **Client library for file access**
  - Talks to master to find chunk servers
  - Connects directly to chunk servers to access data



Distributed File System

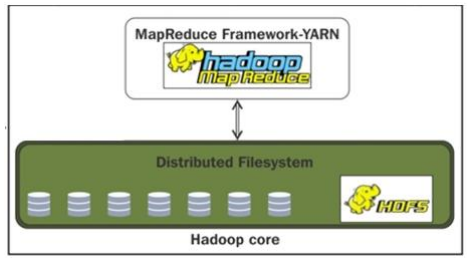
- **Reliable distributed file system**
- Data kept in "chunks" spread across machines
- Each chunk replicated on different machines
  - Seamless recovery from disk or machine failure

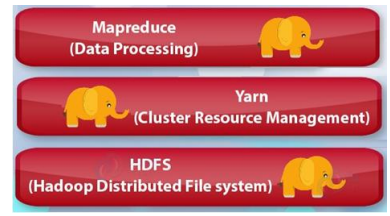


Chunk servers also serve as compute servers

Bring computation directly to the data!

Hadoop alternative





25

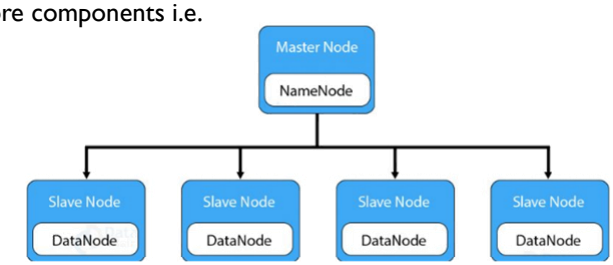
Hadoop Distributed File System (HDFS)

- designed to run on commodity hardware
- HDFS is highly fault-tolerant and is designed to be deployed on low-cost hardware
- HDFS provides high throughput access to application data and is suitable for applications that have large data sets.
- HDFS relaxes a few POSIX requirements to enable streaming access to file system data
- HDFS was originally built as infrastructure for the Apache Nutch web search engine project.

26

Hadoop HDFS

- HDFS is the primary or major component of Hadoop
- responsible for storing large datasets of structured or unstructured data across various nodes
- maintain the metadata in the form of log files
- has a Master-slave architecture
- consists of two core components i.e.
  - NameNode
  - DataNode



28

Hadoop HDFS

NameNode daemon

- runs on the master server
- responsible for Namespace management and regulates file access by the client
- manages modifications to file system namespace
- keeps track of mapping of blocks to DataNodes
- contains metadata (data about data) requiring comparatively fewer resources than the datanodes (that stores data)

DataNode daemon

- runs on slave nodes
- responsible for storing actual business data
- offers actions like open, close and rename files or directories
- serves read/write requests from the file system's client
- creates, deletes and replicates blocks on demand from NameNode
- can use commodity hardware in the distributed environment

Internally, a file is split into several data blocks and stored on a group of datanodes

29

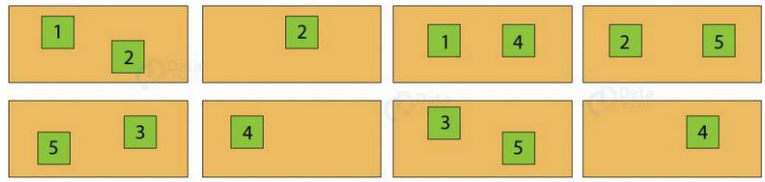
Hadoop HDFS - Block replication



Block Replication

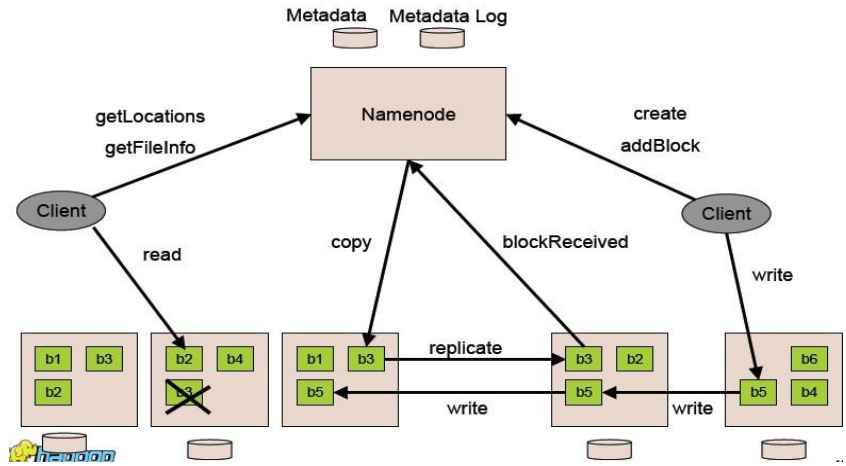
Namenode (Filemane, numReplicas, block-ids, ...)
/user/dataflair/hdata/part-0, r:2, {1,3}, ...
/user/dataflair/hdata/part-1, r:3, {2,4,5}, ...

Datanodes



30

Hadoop HDFS

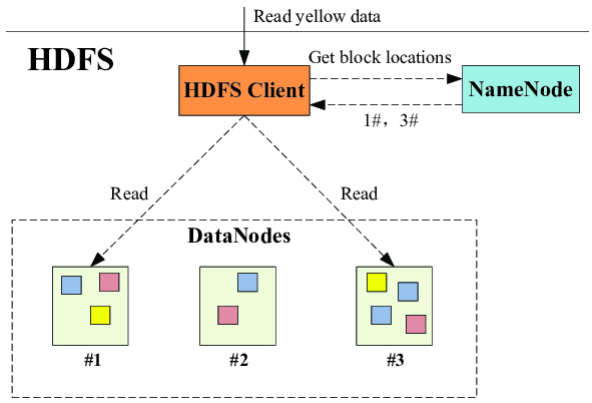


31

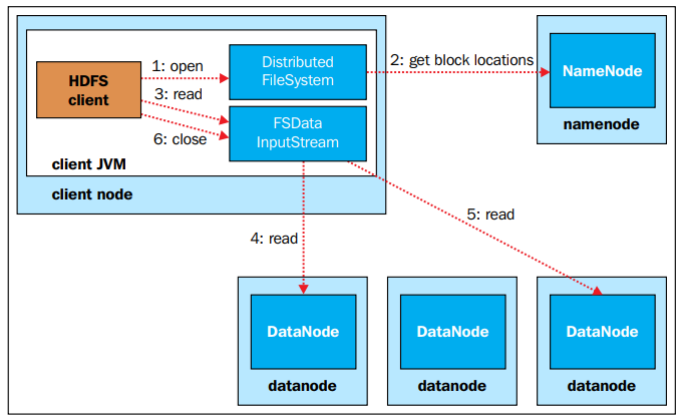
Hadoop HDFS

- NameNode and DataNode communication: Heartbeats
- a DataNode send heartbeats to the NameNode to
  - confirm that the DataNode is operating
  - and the block replicas it hosts are available
- NameNode does not directly call DataNodes
  - It uses replies to heartbeats to send instructions to the DataNodes
  - The instructions include commands to:
    - replicate blocks to other nodes
    - copy data to local
    - remove local block replicas
    - register or shutdown the node
- when a DataNode dies, the NameNode will notice and instruct other DataNodes to replicate data to other DataNodes

HDFS - Get a File

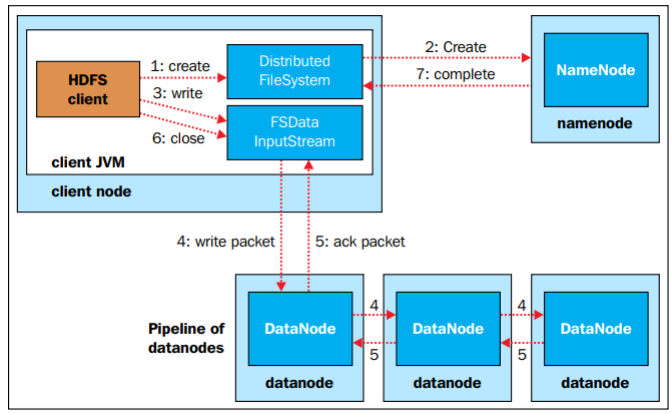


HDFS - Get a File



35

HDFS - Put a file



37

Hadoop HDFS commands

Similar with *nix like commands, but with the prefix `hdfs dfs –`

*nix command	HDFS command
cat	hdfs dfs -cat
chgrp	hdfs dfs -chgrp
chmod	hdfs dfs -chmod
chown	hdfs dfs -chown
cp	hdfs dfs -cp
df	hdfs dfs -df
du	hdfs dfs -du
head	hdfs dfs -head
help	hdfs dfs -help
ls	hdfs dfs -ls
mkdir	hdfs dfs -mkdir
rm	hdfs dfs -rm
rmdir	hdfs dfs -rmdir
tail	hdfs dfs -tail
touch	hdfs dfs -touch

Hadoop HDFS commands

```
hdfs dfs [generic options]
        [-cat [-ignoreCrc] <src> ...]
        [-checksum [-v] <src> ...]
        [-chgrp [-R] GROUP PATH...]
        [-chmod [-R] <MODE[,MODE]... | OCTALMODE> PATH...]
        [-chown [-R] [OWNER][:[GROUP]] PATH...]
        [-concat <target path> <src path> <src path> ...]
        [-copyFromLocal [-f] [-p] [-l] [-d] [-t <thread count>]
                        [-q <thread pool queue size>] <localsrc> ... <dst>]
        [-copyToLocal [-f] [-p] [-crc] [-ignoreCrc] [-t <thread count>]
                        [-q <thread pool queue size>] <src> ... <localdst>]
        [-count [-q] [-h] [-v] [-t [<storage type>]] [-u] [-x] [-e] [-s] <path> ...]
        [-cp [-f] [-p | -p[topax]] [-d] [-t <thread count>]
                        [-q <thread pool queue size>] <src> ... <dst>]
        [-df [-h] [<path> ...]]
        [-du [-s] [-h] [-v] [-x] <path> ...]
        [-expunge [-immediate] [-fs <path>]]
        [-find <path> ... <expression> ...]
        [-get [-f] [-p] [-crc] [-ignoreCrc] [-t <thread count>]
                        [-q <thread pool queue size>] <src> ... <localdst>]
        [-getmerge [-nl] [-skip-empty-file] <src> <localdst>]
```

https://hadoop.apache.org/docs/r3.4.1/hadoop-project-dist/hadoop-hdfs/HDFSCommands.html

Hadoop HDFS commands

```
hdfs dfs [generic options]
        [-head <file>]
        [-help [cmd ...]]
        [-ls [-C] [-d] [-h] [-q] [-R] [-t] [-S] [-r] [-u] [-e] [<path> ...]]
        [-mkdir [-p] <path> ...]
        [-moveFromLocal [-f] [-p] [-l] [-d] <localsrc> ... <dst>]
        [-moveToLocal <src> <localdst>]
        [-mv <src> ... <dst>]
        [-put [-f] [-p] [-l] [-d] [-t <thread count>]
                  [-q <thread pool queue size>] <localsrc> ... <dst>]
        [-rm [-f] [-r|-R] [-skipTrash] [-safely] <src> ...]
        [-rmdir [--ignore-fail-on-non-empty] <dir> ...]
        [-setrep [-R] [-w] <rep> <path> ...]
        [-stat [format] <path> ...]
        [-tail [-f] [-s <sleep interval>] <file>]
        [-text [-ignoreCrc] <src> ...]
        [-touch [-a] [-m] [-t TIMESTAMP (yyyyMMdd:HHmmss) ] [-c] <path> ...]
        [-touchz <path> ...]
        [-truncate [-w] <length> <path> ...]
```


41

Hadoop HDFS commands

Show commands

```
hdfs dfs
```

```
root@0f6491c627f5:/# hadoop fs
Usage: hadoop fs [generic options]
        [-appendToFile <localsrc> ... <dst>]
        [-cat [-ignoreCrc] <src> ...]
        [-checksum <src> ...]
        [-chgrp [-R] GROUP PATH...]
        [-chmod [-R] <MODE[,MODE]... | OCTALMODE> PATH...]
        [-chown [-R] [OWNER][:[GROUP]] PATH...]
        [-copyFromLocal [-f] [-p] [-l] [-d] [-t <thread count>] <localsrc> ... <dst>]
        [-copyToLocal [-f] [-p] [-ignoreCrc] [-crc] <src> ... <localdst>]
        [-count [-q] [-h] [-v] [-t [<storage type>]] [-u] [-x] [-e] <path> ...]
        [-cp [-f] [-p | -p[topax]] [-d] <src> ... <dst>]
        [-createSnapshot <snapshotDir> [<snapshotName>]]
        [-deleteSnapshot <snapshotDir> <snapshotName>]
        [-df [-h] [<path> ...]]
        [-du [-s] [-h] [-v] [-x] <path> ...]
```

***nix like commands with the prefix**

```
hdfs dfs -XCMDX XARGSX
```

or

```
hadoop fs -XCMDX XARGSX
```


44

Hadoop HDFS commands

Create a HDFS directory

```
hdfs dfs -mkdir /input
```

```
root@9ffb9b946aa8:/# hdfs dfs -mkdir  /input
root@9ffb9b946aa8:/#
```

List the contents of a HDFS directory

```
hdfs dfs -ls /
```

```
root@9ffb9b946aa8:/# hdfs dfs -ls  /
Found 2 items
drwxr-xr-x   - root supergroup          0 2021-04-08 18:10 /input
drwxr-xr-x   - root supergroup          0 2021-04-08 18:08 /rmstate
root@9ffb9b946aa8:/#
```

Upload a file into HDFS – put or copyFromLocal

```
hdfs dfs -put ../input/100-00.txt /input
hdfs dfs -copyFromLocal ../input/100-00.txt /input
```

```
root@9ffb9b946aa8:/hdfs dfs -put ../input/100-0.txt /input/
root@9ffb9b946aa8:/# hdfs dfs -copyFromLocal ../input/100-0.txt /input/
copyFromLocal: `/input/100-0.txt': File exists
```


Hadoop HDFS commands

remove a HDFS directory

```
hdfs dfs -rmdir /input
```

```
root@9ffb9b946aa8:/# hdfs dfs -rmdir /input
root@9ffb9b946aa8:/#
```

Remove a file

```
hdfs dfs -rm /input/100-00.txt
```

```
root@9ffb9b946aa8:/# hdfs dfs -rm /input/100-0.txt
Deleted /input/100-0.txt
```

Download a file from HDFS – get or copyToLocal

```
hdfs dfs -get         /input/100-00.txt 100-00.txt1
hdfs dfs -copyToLocal /input/100-00.txt 100-00.txt1
```

```
:/# hdfs dfs -copyToLocal /input/100-0.txt /input/100-0.txt1
:/#
```

Hadoop HDFS commands

See contents of a file in HDFS - cat

```
hdfs dfs -cat /input/100-00.txt
```

```
root@9ffb9b946aa8:/# hdfs dfs -rmdir /input
root@9ffb9b946aa8:/#
```

Copy a file from source to destination in HDFS - cp

```
hdfs dfs -cp /input/100-00.txt /input/100-00.txt2
```

```
root@9ffb9b946aa8:/# hdfs dfs -cp /input/100-0.txt /input/100-0.txt2
root@9ffb9b946aa8:/#
```

Move file from source to destination in HDFS - mv

```
hdfs dfs -mv /input/100-00.txt2 /input/100-00.txt4
```

```
root@9ffb9b946aa8:/# hdfs dfs -mv /input/100-0.txt2 /input/100-0.txt3
root@9ffb9b946aa8:/#
```


Hadoop HDFS commands

Display last few lines of a file (1kb)in HDFS - tail

```
hdfs dfs -tail /input/100-00.txt
```

```
root@9ffb9b946aa8:/# hdfs dfs -tail /input/100-0.txt
tenberg.org/donate
```

Display the size of files and directories in HDFS - du

```
hdfs dfs -du /input/
```

```
root@9ffb9b946aa8:/# hdfs dfs -du  /input/
5757115  17271345  /input/100-0.txt
5757115  17271345  /input/100-0.txt3
```

Help for an individual HDFS command - usage

```
hdfs dfs -usage du
```

```
root@9ffb9b946aa8:/# hdfs dfs -usage du
Usage: hadoop fs [generic options] -du [-s] [-h] [-v] [-x] <path> ...
root@9ffb9b946aa8:/#
```

Hadoop HDFS commands

Display last few lines of a file (1kb)in HDFS - tail

```
hdfs dfs –tail hdfs://10.84.129.52:9000/datasets/books/shakespeare.txt
```

Display the size of files and directories in HDFS - du

```
hdfs dfs -du hdfs://10.84.129.52:9000/datasets
```

Help for an individual HDFS command - usage

```
hdfs dfs –help du
```

```
root@hdfsclient:/bigdata# hdfs dfs –help du
-du [-s] [-h] [-v] [-x] <path> ... :
  Show the amount of space, in bytes, used by the files that match the specified
  file pattern. The following flags are optional:

  -s  Rather than showing the size of each individual file that matches the
      pattern, shows the total (summary) size.
  -h  Formats the sizes of files in a human-readable fashion rather than a number
      of bytes.
  -v  option displays a header line.
  -x  Excludes snapshots from being counted.

  Note that, even without the -s option, this only shows size summaries one level
  deep into a directory.

  The output is in the form
        size    disk space consumed     name(full path)
```


Hadoop HDFS commands

Check the filesystem

```
hdfs fsck /
```

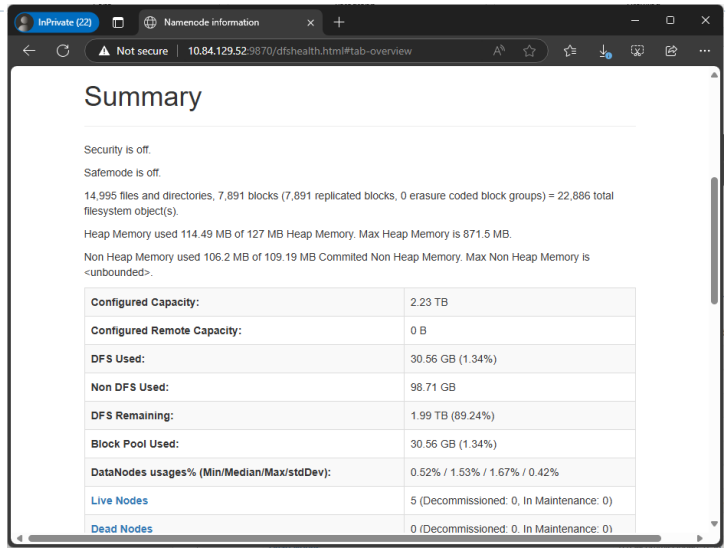
List the corrupted file blocks

```
hdfs fsck -list-corruptfileblocks
```

Delete the corrupted file blocks

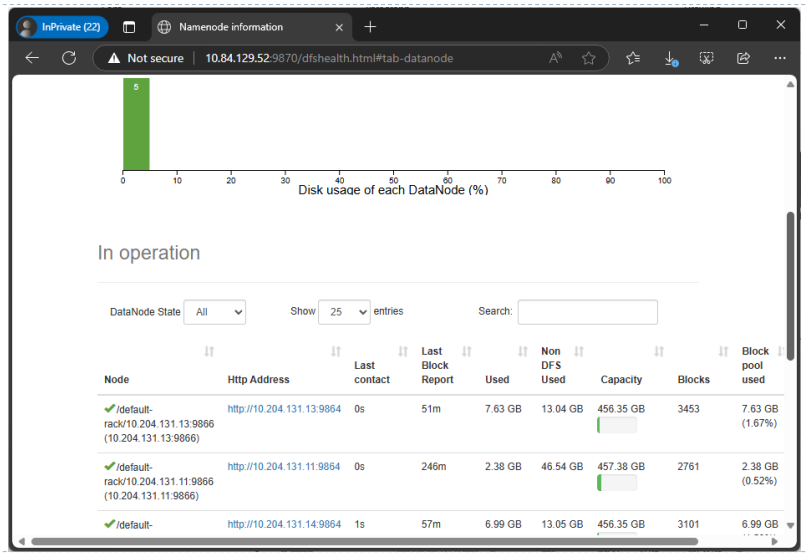
```
hdfs fsck / -delete
```

HDFS



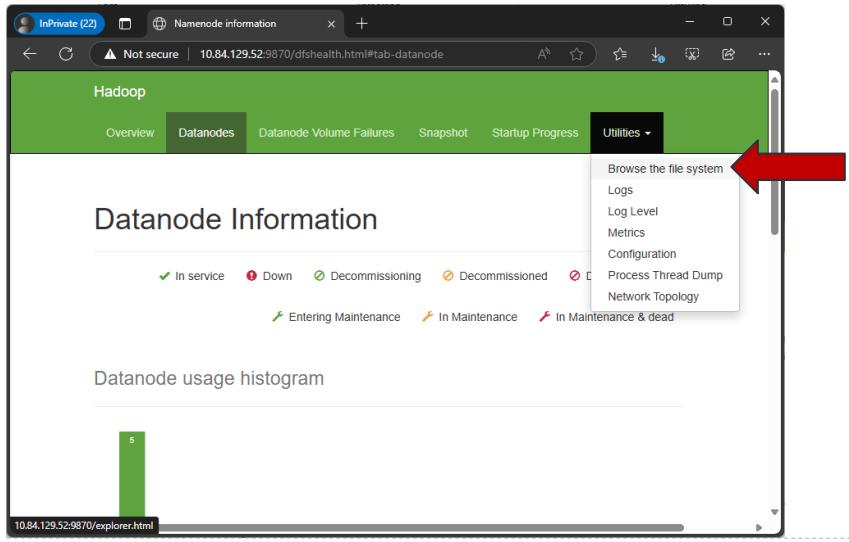
51

HDFS



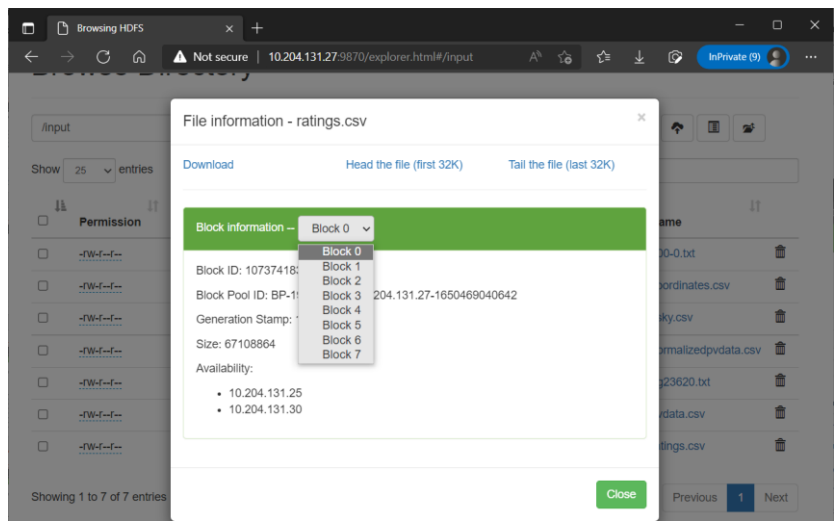
52

HDFS



53

HDFS



54

For worksheet #01

Download the Docker container from

http://10.204.131.100/bigdata/docker_jupyterhive.zip

unzip docker_jupyterhive.zip

cd docker_jupyterhive

In Windows, run
make

In Linux, run
make

```
# Connect to the running container to use as a hdfs client
docker exec –it  <dockerid> bash

# To get the container ID <dockerid> run
docker ps
```

```
# show the existing files on root folder
hdfs dfs –ls /
```